

Name: Andrew Scott

UHID: 15991670

Homework #01: Numbering Systems

1. Convert the $b_2 \rightarrow b_{16}$ binary numbers (unsigned) to decimal:
- 1) $0b11010100$; 2) $0b01011010$; 3) $0b00111100$; 4) $0b10101011$;
- $1(2^7) + 2^6 + 2^4 + 2^2 = 128 + 64 + 16 + 4 = 212_{10}$
 $64 + 16 + 8 + 2 = 90_{10}$
 $32 + 16 + 8 + 4 = 60_{10}$
 $128 + 32 + 8 + 2 + 1 = 171_{10}$

2. Convert the $b_{10} \rightarrow b_2$ decimal numbers to binary:
- 1) 23; 2) 127; 3) 37; 4) 106. 1101010
- $2 \overline{) 23} \rightarrow 1$
 $2 \overline{) 11} \rightarrow 1$
 $2 \overline{) 5} \rightarrow 1$
 $2 \overline{) 2} \rightarrow 0$
 $1 \rightarrow 1$
 $0 \rightarrow 0$
- $2 \overline{) 127} \rightarrow 1$
 $2 \overline{) 63} \rightarrow 1$
 $2 \overline{) 31} \rightarrow 1$
 $2 \overline{) 15} \rightarrow 1$
 $2 \overline{) 7} \rightarrow 1$
 $2 \overline{) 3} \rightarrow 1$
 $2 \overline{) 1} \rightarrow 1$
 $0 \rightarrow 0$
- $2 \overline{) 37} \rightarrow 1$
 $2 \overline{) 18} \rightarrow 0$
 $2 \overline{) 9} \rightarrow 1$
 $2 \overline{) 4} \rightarrow 0$
 $2 \overline{) 2} \rightarrow 0$
 $1 \rightarrow 1$
 $0 \rightarrow 0$
- $2 \overline{) 106} \rightarrow 0$
 $2 \overline{) 53} \rightarrow 1$
 $2 \overline{) 26} \rightarrow 0$
 $2 \overline{) 13} \rightarrow 1$
 $2 \overline{) 6} \rightarrow 0$
 $2 \overline{) 3} \rightarrow 1$
 $1 \rightarrow 1$

3. Convert the hexadecimal numbers (unsigned) to decimal:
- 1) $0xE2$; 2) $0x9A$; 3) $0x3722$; 4) $0xA282$.
- $14 \times 16 + 2 \times 1 = 226_{10}$
 $9 \times 16 + 10 = 154_{10}$
 $12,288 + 1,792 + 32 + 2 = 14,114_{10}$
 $40,960 + 5,120 + 1,280 + 2 = 47,362_{10}$

4. Convert the decimal numbers to hexadecimal:
- 1) 4096; 2) 832; 3) 1927; 4) 321.
- $16 \overline{) 4096} \rightarrow 0$
 $16 \overline{) 256} \rightarrow 6$
 $16 \overline{) 16} \rightarrow 0$
 $1 \rightarrow 1$
- $16 \overline{) 832} \rightarrow 5$
 $16 \overline{) 52} \rightarrow 4$
 $16 \overline{) 3} \rightarrow 3$
- $16 \overline{) 1927} \rightarrow 7$
 $16 \overline{) 120} \rightarrow 8$
 $16 \overline{) 7} \rightarrow 7$
- $16 \overline{) 321} \rightarrow 1$
 $16 \overline{) 20} \rightarrow 4$
 $16 \overline{) 1} \rightarrow 1$
- $0x1000$
 $0x340$
 $0x187$
 $0x141$

5. Convert hexadecimal to binary and convert binary to hexadecimal
- 1) $0b00111100$; 2) $0b10100011$; 3) $0x9A$; 4) $0x3722$.
- $0x3C$
 $0xA3$
 $0x10011010$
 $0x00110110010010$



6. Convert the numbers into the specified number system (If you need a digit out of the range of 0-9 and A-F, circle the number to create a digit. For example, you

can use $\textcircled{21}$ to represent the number 21.):

1) Convert $(1023)_5$ to base-8 (Radix = 8); *convert to base 10, then convert to base 8*

2) Convert $(762)_8$ to base-5;

3) Convert $(5\textcircled{18})_{20}$ to base-60; *this means base 20 for the digits 5 and 18*
 $(5^{18})_{20} \rightarrow 5 \times 20^1 + 18 \times 20^0 \rightarrow 100 + 18 \rightarrow 118_{10} \xrightarrow{\text{base 60}} 60 \overline{) 118} \rightarrow 58$
 $\frac{1}{0} \rightarrow 1$
 $\textcircled{58}_{60}$

4) Convert $(A1)_{12}$ to base-20.

1) 510
 1023

$1(12^2) + 2(12^1) + 3(12^0) = 138_{10}$
 $8 \overline{) 138} \rightarrow 2 \quad 212_8$
 $\frac{17}{0} \rightarrow 1$
 $\frac{2}{0} \rightarrow 2$

2) 210
 $762_8 \rightarrow 65$
 $7(8^2) + 6(8^1) + 2(8^0) = 498_{10}$
 $448 + 48 + 2$
 $6 \overline{) 498} \rightarrow 3 \quad 3443_5$
 $\frac{19}{0} \rightarrow 4$
 $\frac{19}{3} \rightarrow 4$
 $\frac{3}{0} \rightarrow 3$

4) 10
 $A1_{12} \rightarrow \text{base 20}$
 $10(12^1) + 1(12^0) = 121_{10}$
 $20 \overline{) 121} \rightarrow 6$
 $\frac{61}{0} \rightarrow 1$
 $\textcircled{61}_{20}$

7. Change the negative decimal numbers into signed 8-bit binary, using all three representations including sign-magnitude, ones' complement, and two's complement:

1) 0;

sign-mag
 00000000
 10000000
 $1's \text{ comp.}$
 00000000
 11111111
 $2's \text{ comp.}$
 00000000

2) -1;

sign-mag
 10000001
 $1's \text{ comp.}$
 11111110
 $2's \text{ comp.}$
 11111111

3) -79;

$2179 \rightarrow 1$
 $39 \rightarrow 1$
 $19 \rightarrow 1$
 $9 \rightarrow 1$
 $4 \rightarrow 0$
 $2 \rightarrow 0$
 $1 \rightarrow 1$
 $0 \rightarrow 1$
 $\textcircled{0} \rightarrow 1$
will make the binary neg.

4) -63

sign-mag
 11001111
 $1's \text{ comp.}$
 10110000
 $2's \text{ comp.}$
 10110001
 $2163 \rightarrow 1$
 $31 \rightarrow 1$
 $15 \rightarrow 1$
 $7 \rightarrow 1$
 $3 \rightarrow 1$
 $1 \rightarrow 0$
 $0 \rightarrow 1$
 $15 \times 2 = 1.5$
 $2's \text{ comp.}$
 1100001

8. To add two 8-bit signed binary numbers using two's complement, simply align the values on the least significant bit and add, propagating any carry to the bit one position high. If you have a carry past the end of the word (overflow), just drop it. Convert the numbers in the following into signed binary using two's complement, add them, and convert it back to decimal. Check if the outputs are correct, if not,

explain why.

1) $-23+38$;

2) $23 + (-38)$;

3) $67+82$;

4) $(-63) + (-72)$

$$\begin{array}{r} 2 \overline{) 23} \rightarrow 1 \\ \underline{11} \rightarrow 1 \\ \underline{5} \rightarrow 1 \\ \underline{2} \rightarrow 0 \\ \underline{1} \rightarrow 1 \\ \underline{0} \rightarrow 1 \end{array}$$

$$\begin{array}{r} 2 \overline{) 38} \rightarrow 0 \quad 00100110 \\ \underline{14} \rightarrow 1 \\ 9 \rightarrow 1 \\ \underline{4} \rightarrow 0 \\ 2 \rightarrow 0 \\ \underline{1} \rightarrow 1 \\ 0 \rightarrow 0 \end{array}$$

$00010111 \quad 23$
 $00100110 \quad -38$
 11011001
 $11011010 \leftarrow -38$
 00010111
 11011010

 $11110010_2 \leftarrow -19_{10}$

$$\begin{array}{r} 2 \overline{) 67} \rightarrow 1 \\ \underline{33} \rightarrow 1 \\ 16 \rightarrow 0 \\ \underline{8} \rightarrow 0 \\ 4 \rightarrow 0 \\ \underline{2} \rightarrow 0 \\ 1 \rightarrow 1 \\ \underline{0} \rightarrow 0 \end{array}$$

$2 \overline{) 63} \rightarrow 1$
 $\underline{31}$
 $15 \rightarrow 1$
 $\underline{7}$
 $3 \rightarrow 1$
 $\underline{1}$
 $0 \rightarrow 0$

$$\begin{array}{r}
 + 11000001 \\
 10111000 \\
 \hline
 18111001 \quad \uparrow \\
 -128_{10}
 \end{array}$$

$$\begin{array}{r}
 -63 \\
 -72 \\
 \hline
 -135_{10}
 \end{array}$$

↑ this is overflowing

$$\begin{array}{r} 2 \overline{) 82} \rightarrow 0 \\ 41 \rightarrow 1 \\ \hline 20 \rightarrow 0 \\ 10 \rightarrow 0 \\ \hline 5 \rightarrow 1 \\ \hline 2 \rightarrow 0 \\ \hline 1 \rightarrow 1 \\ \hline 0 \rightarrow 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 72} \rightarrow 0 \\ \underline{36} \rightarrow 0 \\ 18 \rightarrow 0 \\ \underline{9} \rightarrow 1 \\ 4 \rightarrow 0 \\ \underline{2} \rightarrow 0 \\ 1 \rightarrow 1 \\ \underline{0} \rightarrow 0 \end{array}$$

$$\begin{array}{r} 01000011 \cdot 67 \\ 01010010 \cdot 82 \\ \hline 100101012 \end{array} \rightarrow$$

$1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 = 149$ ← overflow
 The max positive 8 bit is 127

also, the sign notation indicates that the number is negative, which is incorrect so the direction that the digit number is leading.

9. (Extra 10 pts, use additional sheets) Using any language that you prefer, e.g. C/C++, Python, Matlab, Java, etc., to develop a function which can convert your people soft number into a positioned number system base from 2 to 20. Use A to J to represent the digit 10 to 19. The radix can be defined as a parameter. You can either past your code or describe your algorithm here. Then show me 3 examples of the inputs and outputs of your function.
10. (Extra 10 pts, use additional sheets) If the processing unit of the processor has M positions (which is called bits in common computers), and each position has N states (radices), it can represent N^M (N to the power of M) states. That means the processor can represent the number from 0 to N^M . Let's assume the cost to build the processor is proportional to the number of the states, which are $M \cdot N \cdot C_0$, where C_0 is the cost of a state; For example, an 8-positioned base-2 unit can represent the numbers from 0 to 255 (2^8) with a cost of $16C_0$ ($2 \cdot 8$); a 5-positioned base-3 unit can represent the numbers from 0 to 242 (3^5) with a cost of $15C_0$ ($3 \cdot 5$). The ratios of the representable numbers and the cost are $(256/16C_0)$ and $(243/15C_0)$, respectively. Please consider:
- 1) If we want to have a processor capable of handling the number from 0 to 99,999. How many positions does the processing unit need if the unit is based on the radices of 2, 3, 5 or 8?
 - 2) Compare the ratio of the total numbers and costs for each case in 1).
 - 3) If the number goes to $[0, \infty)$, which base has the highest number to cost ratio?

9. (Extra 10 pts, use additional sheets) Using any language that you prefer, e.g. C/C++, Python, Matlab, Java, etc., to develop a function which can convert your people soft number into a positioned number system base from 2 to 20. Use A to

J to represent the digit 10 to 19. The radix can be defined as a parameter. You can either past your code or describe your algorithm here. Then show me 3 examples of the inputs and outputs of your function.

```
def convert_base(num, base):
    if base < 2 or base > 20:
        raise ValueError("Base must be between 2 and 20")
    if num == 0:
        return "0"
    digits = "0123456789ABCDEFGHIJ"
    result = []
    n = abs(num)
    while n > 0:
        remainder = n % base
        result.append(digits[remainder])
        n //= base
    if num < 0:
        result.append('-')
    return "".join(reversed(result))
```

```
ps_id = 1599670
print(f"PeopleSoft ID: {ps_id}")
print(f"Base 2: {convert_base(ps_id, 2)}")
print(f"Base 8: {convert_base(ps_id, 8)}")
print(f"Base 16: {convert_base(ps_id, 16)}")
print(f"Base 20: {convert_base(ps_id, 20)}")
```

Out put:

```
PeopleSoft ID: 1599670
Base 2: 110000101101010110110
Base 8: 6056533
Base 16: 186B86
Base 20: 9JJ3A
```

10. (Extra 10 pts, use additional sheets) If the processing unit of the processor has M positions (which is called bits in common computers), and each position has N states (radices), it can represent N^M (N to the power of M) states. That means the processor can represent the number from 0 to N^M . Let's assume the cost to build the processor is proportional to the number of the states, which are $M \cdot N \cdot C_0$,

where C_0 is the cost of a state; For example, an 8-positioned base-2 unit can represent the numbers from 0 to 255 (2^8) with a cost of $16C_0$ (2^8); a 5-positioned base-3 unit can represent the numbers from 0 to 242 (3^5) with a cost of $15C_0$ (3^5). The ratios of the representable numbers and the cost are $(256/16C_0)$ and $(243/15C_0)$, respectively. Please consider:

- 1) If we want to have a processor capable of handling the number from 0 to 99,999. How many positions does the processing unit need if the unit is based on the radices of 2, 3, 5 or 8?
- 2) Compare the ratio of the total numbers and costs for each case in 1).
- 3) If the number goes to $[0, \infty)$, which base has the highest number to cost ratio?

Let total representable range: 0 to $N^M - 1 \rightarrow$ max number is $N^M - 1$.

We want **max number $\geq 99,999$** (so $N^M - 1 \geq 99999$, i.e., $N^M \geq 100000$).

Cost = $M \times N \times C_0$ (states per position $\times$ positions $\times$ constant).

Ratio = $\frac{N^M}{MNC_0} = \frac{N^{M-1}}{MC_0}$ (since C_0 is constant, compare $\frac{N^{M-1}}{M}$ for best efficiency ignoring C_0).

1) Find M for each N so that $N^M \geq 100000$

- **Base 2 ($N = 2$):**
 $2^M \geq 100000 \rightarrow M \geq \log_2(100000) \approx 16.61 \rightarrow M = 17$.
Max number: $2^{17} = 131072$
Cost factor: $M \times N = 17 \times 2 = 34$
- **Base 3 ($N = 3$):**
 $3^M \geq 100000 \rightarrow M \geq \log_3(100000) \approx 10.48 \rightarrow M = 11$
Max number: $3^{11} = 177147$
Cost: $11 \times 3 = 33$
- **Base 5 ($N = 5$):**
 $5^M \geq 100000 \rightarrow M \geq \log_5(100000) \approx 7.15 \rightarrow M = 8$
Max number: $5^8 = 390625$
Cost: $8 \times 5 = 40$
- **Base 8 ($N = 8$):**
 $8^M \geq 100000 \rightarrow M \geq \log_8(100000) \approx 5.54 \rightarrow M = 6$
Max number: $8^6 = 262144$
Cost: $6 \times 8 = 48$

So:

Base 2: $M=17$, Cost factor=34

Base 3: $M=11$, Cost factor=33

Base 5: $M=8$, Cost factor=40

Base 8: $M=6$, Cost factor=48

2) Compare ratio $\frac{N^M}{MNC_0}$

Ignore C_0 (common factor), compute $\frac{N^{M-1}}{M}$:

- $N=2, M=17: \frac{2^{16}}{17} = \frac{65536}{17} \approx 3855.06$
- $N=3, M=11: \frac{3^{10}}{11} = \frac{59049}{11} \approx 5368.09$
- $N=5, M=8: \frac{5^7}{8} = \frac{78125}{8} = 9765.625$
- $N=8, M=6: \frac{8^5}{6} = \frac{32768}{6} \approx 5461.33$

Order by ratio:

Base 5 (9765.6) > Base 8 (5461.3) > Base 3 (5368.1) > Base 2 (3855.1)

So **Base 5 gives best number-to-cost ratio** for covering up to 100,000.

3) For $[0, \infty)$, best base as $M \rightarrow \infty$

We want to maximize $\frac{N^{M-1}}{M}$ for given N, M satisfying $N^M \geq R$ (R large).

As $R \rightarrow \infty$, the optimal N for fixed large M is found by maximizing $\frac{N^{M-1}}{M}$.

Actually, take log of ratio per cost:

$$\ln \left(\frac{N^{M-1}}{M} \right) = (M-1) \ln N - \ln M.$$

But M depends on N for fixed R : $M \approx \frac{\ln R}{\ln N}$.

Plug $M = \frac{\ln R}{\ln N}$ into $(M-1) \ln N - \ln M$:

$$(M-1) \ln N = \left(\frac{\ln R}{\ln N} - 1 \right) \ln N = \ln R - \ln N$$

So expression becomes:

$$\ln \left(\frac{N^{M-1}}{M} \right) = \ln R - \ln N - \ln \left(\frac{\ln R}{\ln N} \right)$$

For large R , $\ln R$ is constant, so maximize $-\ln N - \ln \left(\frac{1}{\ln N} \right)$? Wait carefully:

$$-\ln N - \ln \left(\frac{\ln R}{\ln N} \right) = -\ln N - \ln (\ln R) + \ln (\ln N)$$

Derivative wrt N :

$$-\frac{1}{N} + \frac{1}{N \ln N} = \frac{1}{N} \left(\frac{1}{\ln N} - 1 \right)$$

Set to 0 $\rightarrow \frac{1}{\ln N} - 1 = 0 \Rightarrow \ln N = 1 \Rightarrow N = e \approx 2.718$.

Thus **Base e (~2.718)** maximizes efficiency for large R .

But since N must be integer, check $N=3$ vs $N=2$: earlier we saw $N=3$ better than $N=2$ for large numbers.

Thus **in practice, base 3 is best** integer radix for maximal range-to-cost as $R \rightarrow \infty$.

Final answers for Problem 10:

1. $M = 17$ (base 2), $M = 11$ (base 3), $M = 8$ (base 5), $M = 6$ (base 8).
2. Ratios (ignoring C_0): Base 5 highest for up to 100,000.
3. As range $\rightarrow \infty$, optimal base is e , best integer base is 3.